



Concertx

Sistema Interactivo para Conciertos y Eventos

Flutter (Dart) · Express.js · PostgreSQL

Nombre: **Rubén Mendoza Dorantes**
Matrícula: **2023371079**
Materia: **Desarrollo para Dispositivos Inteligentes**
Evaluación: **1**
Fecha: **17 de junio de 2026**

Índice

1. Descripción General del Proyecto	3
2. SA.1 — Funcionalidad de la Aplicación	3
2.1. Descripción del Problema	3
2.2. Objetivo General	3
2.3. Objetivos Específicos	4
2.4. Alcance del Proyecto	5
2.5. Casos de Uso	6
2.6. Datos Manejados	8
2.7. Integraciones Externas	8
2.8. Viabilidad Técnica	8
3. SA.2 — Usabilidad y Accesibilidad	9
3.1. Público Objetivo	9
3.2. Flujo de Usuario Simplificado	9
3.3. Accesibilidad	10
3.4. Diseño Responsivo por Dispositivo	10
3.5. Densidad de Elementos por Dispositivo	10
3.6. Pruebas de Usabilidad Planeadas	12
4. SA.3 — Diseño de Interfaces de Usuario	13
4.1. Paleta de Colores	13
4.2. Tipografía	13
4.3. Componentes Reutilizables	14
4.4. Layout por Dispositivo	16
4.5. Iconografía	17
4.6. Estados de Componentes	19
4.7. Guía de Estilo	19
5. SA.4 — Seguridad Transversal	20
5.1. Leyes Aplicables	21
5.2. Datos Personales Identificados	21
5.3. Justificación de Seguridad	21
5.4. Plan de Almacenamiento	21
5.5. Comunicaciones Seguras	22
5.6. Gestión de Credenciales y API Keys	22
5.7. Validación de Entrada	23

5.8. Aviso de Privacidad	23
5.9. Certificación de Acceso para Pruebas	25

1. Descripción General del Proyecto

Concertx es una aplicación para Android que transforma la experiencia de asistir a un concierto al convertir el teléfono y el smartwatch de cada asistente en parte del espectáculo. Al abrir la aplicación, el usuario encuentra una lista de los conciertos próximos y puede buscar eventos por nombre del artista o por estadio. Desde ahí tiene dos opciones: unirse a un evento existente ingresando el código que le compartieron, o crear su propio diseño de efectos para un evento. Quien crea el diseño puede configurar los colores, vibraciones y animaciones para cada canción del setlist; al terminar, el sistema genera un código único que puede compartir con los demás asistentes para que se unan. Una vez conectados, todos los dispositivos reciben los efectos en tiempo real de forma sincronizada, haciendo que el público entero sea parte del show. La solución está construida con **Flutter y Dart** para el cliente móvil y la interfaz del smartwatch, **Express.js** como backend que gestiona la lógica de negocio y las APIs REST, y **PostgreSQL** como base de datos relacional, con opción de alojarla en **Supabase** o **Microsoft Azure**.

2. SA.1 — Funcionalidad de la Aplicación

2.1. Descripción del Problema

Los conciertos de gran escala invierten cantidades considerables en efectos de escenario —luces, pirotecnia, pantallas LED— pero el público permanece como observador. Las alternativas existentes para involucrar a los asistentes, como pulseras luminosas o aplicaciones de linterna, dependen de hardware costoso o de una coordinación manual que raramente funciona bien en vivo.

El problema concreto es la falta de una herramienta accesible, sin equipos adicionales, que convierta los dispositivos personales del público en parte del espectáculo lumínico. Concertx atiende esa necesidad aprovechando lo que cada asistente ya trae en el bolsillo.

2.2. Objetivo General

Desarrollar una aplicación Android que permita a cualquier usuario crear diseños de efectos visuales y de vibración para un concierto, generar un código de acceso y sincronizar en tiempo real los dispositivos de todos los asistentes que se unan, sin requerir hardware adicional ni un panel de administración externo.

2.3. Objetivos Específicos

1. **Exploración y búsqueda de eventos:** Al iniciar la app, el usuario puede ver la lista de conciertos disponibles o buscar por nombre del artista o estadio.
2. **Creación de diseño de efectos:** El usuario que crea el evento puede configurar colores, animaciones y patrones de vibración para cada canción del setlist. Al finalizar, el sistema genera un código único para compartir.
3. **Unirse a un evento:** Cualquier asistente puede ingresar el código recibido para unirse al evento y comenzar a recibir los efectos en su dispositivo.
4. **Efectos visuales en el teléfono:** La pantalla del dispositivo Android cambia de color, parpadea o despliega animaciones en sincronía con la música y los momentos configurados por el creador del diseño.
5. **Vibración y color en el smartwatch:** El reloj inteligente responde con patrones de vibración y cambios de pantalla según lo programado para cada canción.
6. **Visualización en pantalla inteligente o Smart TV:** Las pantallas conectadas al sistema muestran en tiempo real el mapa del estadio con los asientos activos, el conteo de usuarios sincronizados y el código para unirse al evento. Esta vista funciona como apoyo visual para el recinto sin requerir interacción directa.

2.4. Alcance del Proyecto

- Aplicación móvil para dispositivos **Android** (Flutter/Dart con Android Studio)
 - Pantalla de listado y búsqueda de conciertos por artista o estadio
 - Flujo de **creación de diseño** con efectos por canción y generación de código
 - Flujo de **unirse a evento** mediante código compartido
 - Interfaz para **smartwatch** con efectos visuales y vibración sincronizados
 - Vista para **Smart TV / pantallas inteligentes** (1920×1080 px) con las siguientes funciones específicas:
 - Mapa de asientos del estadio mostrando en tiempo real qué lugares están conectados
 - Contador de usuarios que se han unido al evento
 - Código del evento visible para que nuevos asistentes puedan unirse
 - Modo de solo lectura — la pantalla recibe datos pero no envía ni modifica efectos
 - **Backend con Express.js** para las APIs REST y comunicación en tiempo real
 - **Base de datos PostgreSQL** para usuarios, eventos y configuraciones de efectos
-
- Versión para iOS — la app es exclusivamente Android
 - Venta o validación de boletos de entrada
 - Control de iluminación física del escenario o hardware externo
 - Reproducción o transmisión de audio y video
 - Soporte para dispositivos sin conexión a internet

2.5. Casos de Uso

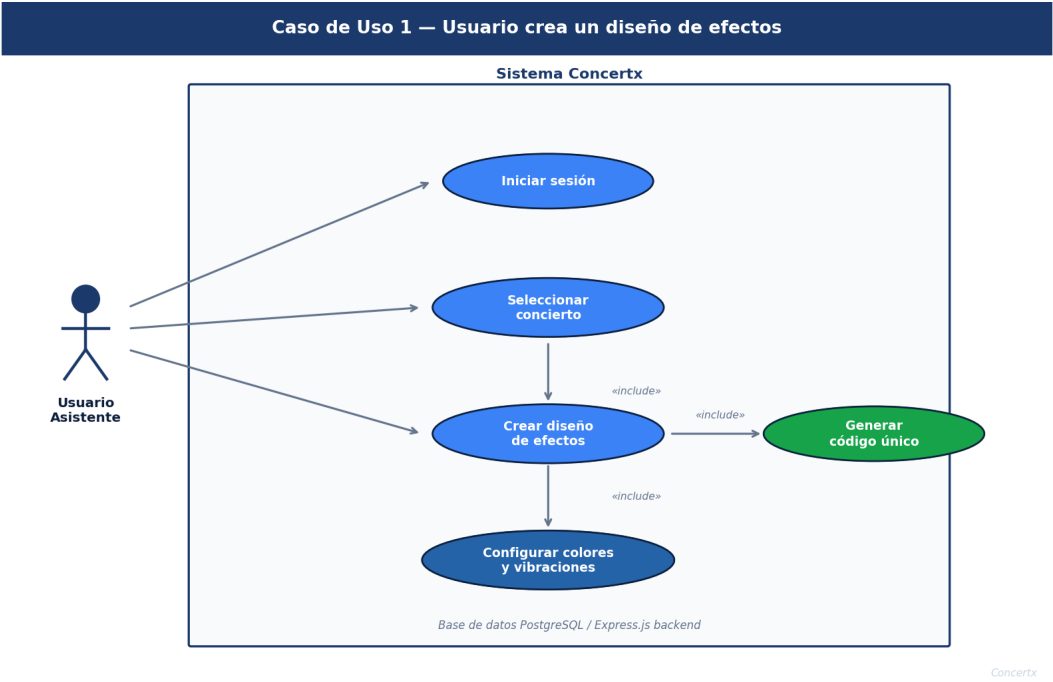


Figura 1: Caso de Uso 1 — Usuario crea un diseño de efectos

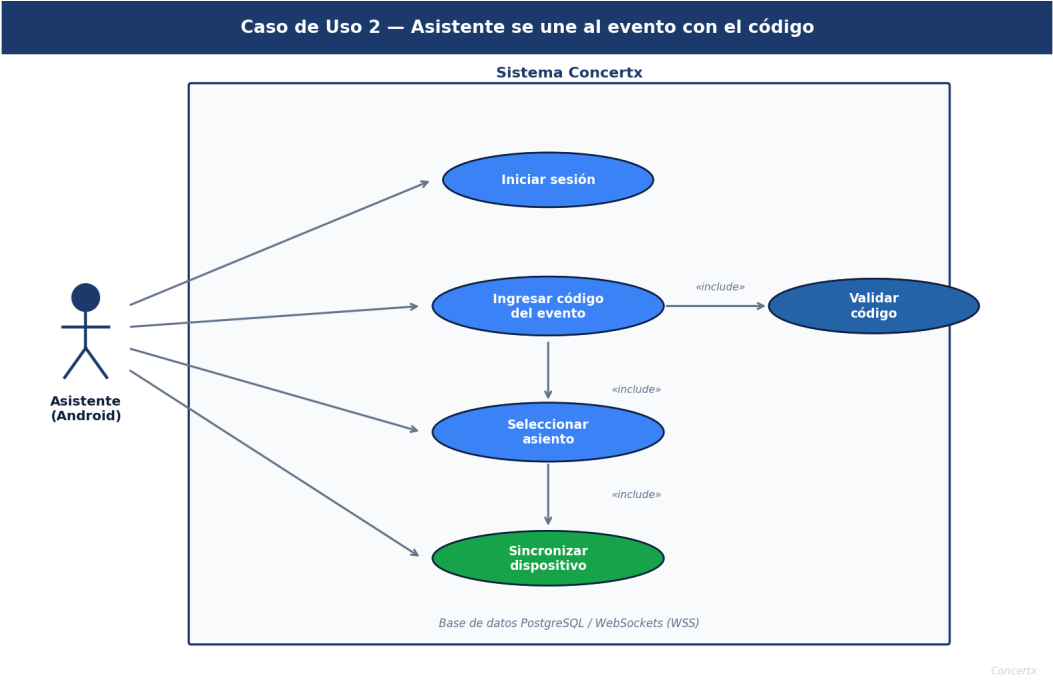


Figura 2: Caso de Uso 2 — Asistente se une al evento con el código

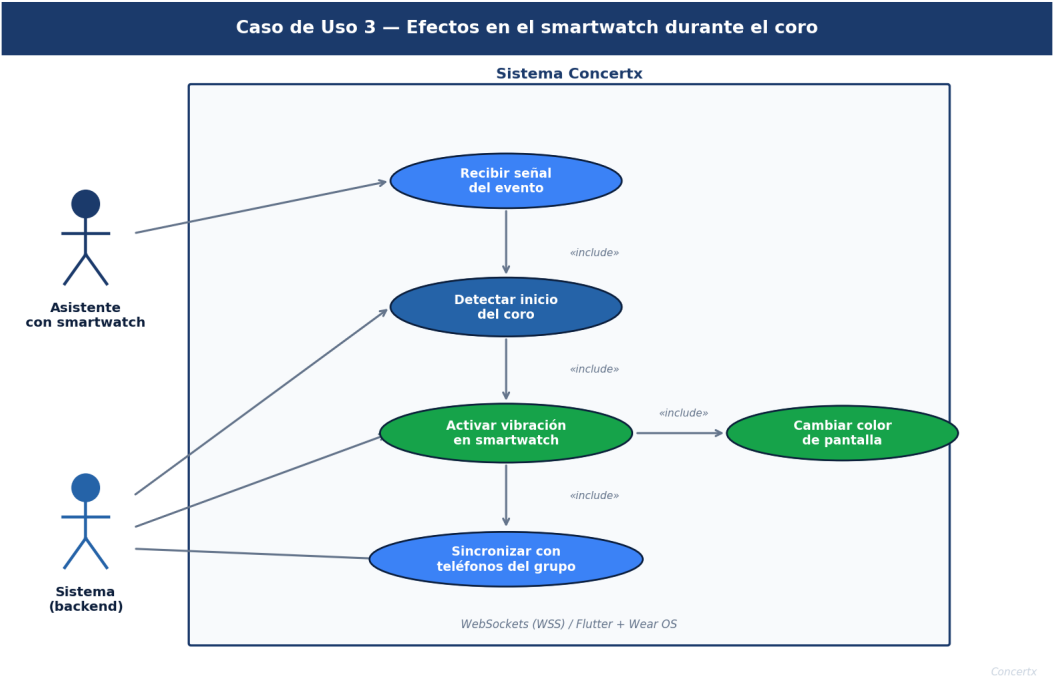


Figura 3: Caso de Uso 3 — Efectos en el smartwatch durante el coro

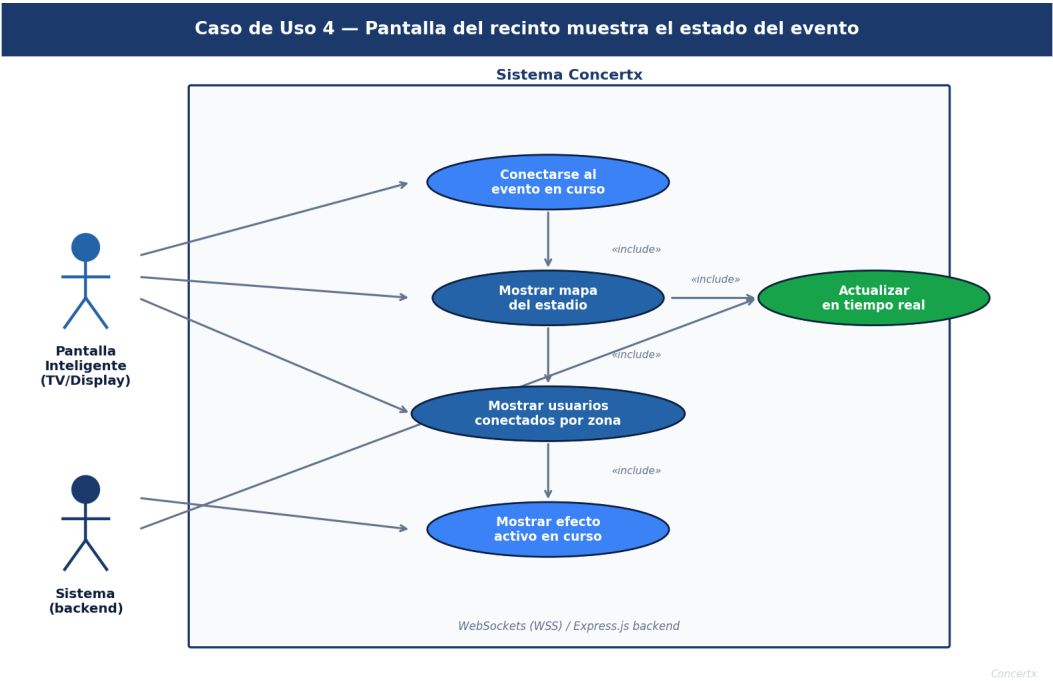


Figura 4: Caso de Uso 4 — Pantalla del recinto muestra el estado del evento

2.6. Datos Manejados

Dato	Descripción	Origen
Nombre completo	Identificación del usuario en la plataforma	Usuario
Correo electrónico	Acceso y recuperación de cuenta	Usuario
Número de teléfono	Dato de contacto registrado al crear la cuenta	Usuario
Contraseña	Autenticación, almacenada con hash bcrypt	Usuario
Código del evento	Cadena alfanumérica generada al crear un diseño	Sistema
Diseño de efectos	Colores, vibraciones y animaciones por canción	Usuario creador
Ubicación en estadio	Zona, fila y número de asiento del asistente	Usuario
Usuarios conectados	Conteo en tiempo real por zona del evento	Sistema

2.7. Integraciones Externas

- **PostgreSQL via Supabase o Azure Database for PostgreSQL** — Almacenamiento relacional de usuarios, eventos, diseños de efectos y asientos. Ambas opciones ofrecen conexión segura mediante SSL y credenciales de entorno.
- **Supabase Auth** (*si se elige Supabase*) — Gestión de registro, inicio de sesión y tokens JWT integrada con la base de datos PostgreSQL.
- **Azure Active Directory / JWT propio** (*si se elige Azure*) — Gestión de identidades y despliegue del backend Express.js en la nube de Microsoft.
- **WebSockets (socket.io)** — Canal de comunicación en tiempo real entre el backend Express.js y los dispositivos, para distribuir efectos con latencia mínima.

2.8. Viabilidad Técnica

El proyecto es técnicamente realizable por las siguientes razones. **Dart y Flutter** permiten construir la aplicación Android y la interfaz del smartwatch desde una sola base de código, con Android Studio como IDE oficial que incluye emuladores para pruebas sin necesidad de dispositivo físico. **Express.js** es un framework ligero y bien documentado para construir APIs REST en Node.js; su integración con `socket.io` facilita la comunicación en tiempo real con múltiples clientes de forma sencilla. **PostgreSQL** es una base de datos relacional madura y adecuada para gestionar usuarios, eventos, diseños de efectos y registros de asientos; tanto Supabase como Azure ofrecen instancias gestionadas con alta disponibilidad y respaldos automáticos. La latencia estimada de WebSockets sobre redes WiFi de evento es inferior a 300 ms, suficiente para que los efectos visuales se perciban como sincronizados por todos los

asistentes.

Subtotal SA.1: ____ / 8 elementos

3. SA.2 — Usabilidad y Accesibilidad

3.1. Público Objetivo

Característica	Descripción
Edad	Entre 15 y 45 años, rango típico de asistentes a conciertos
Experiencia técnica	Básica a media — usuarios habituales de redes sociales y mensajería
Contexto de uso	Ambiente ruidoso, con poca luz, de pie o sentado, manos parcialmente ocupadas
Dispositivos	Teléfono Android propio, smartwatch de forma opcional
Conexión	WiFi del recinto o datos móviles

3.2. Flujo de Usuario Simplificado

El recorrido del asistente desde que llega al evento hasta que participa en los efectos:

1. **Descarga la app** en Google Play antes del evento o mediante código QR en la entrada.
2. **Crea su cuenta** con nombre, correo, número de teléfono y contraseña.
3. **Busca el concierto** por nombre del artista o estadio, o lo selecciona de la lista.
4. **Elige su rol:** crea un diseño de efectos o se une con el código que le compartieron.
5. **Selecciona su asiento:** zona, fila y número.
6. **Disfruta el concierto** — la app responde automáticamente a los efectos programados.

Cada pantalla está diseñada bajo el principio de **dos toques máximo** para completar cualquier acción, ya que el usuario está en un entorno donde no puede prestar atención sostenida a la pantalla.

3.3. Accesibilidad

- **Contraste de colores:** Todo texto sobre fondo oscuro cumple una relación mínima de contraste de 4.5:1, conforme al estándar WCAG 2.1 nivel AA.
- **Área táctil:** Los botones y elementos interactivos tienen un área mínima de 44×44 px para facilitar la interacción con una sola mano.
- **Etiquetas en íconos:** Los íconos relevantes incluyen texto descriptivo visible, sin depender solo del símbolo para comunicar su función.
- **Vibración como canal alternativo:** Los usuarios con discapacidad visual pueden seguir los momentos del concierto a través de los patrones de vibración del smartwatch.
- **Tamaño de texto:** El cuerpo de texto no es inferior a 14 px en ninguna pantalla de la aplicación móvil.

3.4. Diseño Responsivo por Dispositivo

Dispositivo	Cómo se adapta	Resolución
Teléfono Android	Una columna, botones grandes, navegación en barra inferior	360–720 px
Smartwatch	Pantalla completa de color con ícono central, sin menús	384×384 px
Pantalla / TV	Mapa de asientos, contador de usuarios y código del evento	1920×1080 px

3.5. Densidad de Elementos por Dispositivo

- **Teléfono Android:** Máximo 3 elementos por pantalla durante el concierto. La vista principal ocupa el 70 % con el efecto activo; el resto es navegación básica.

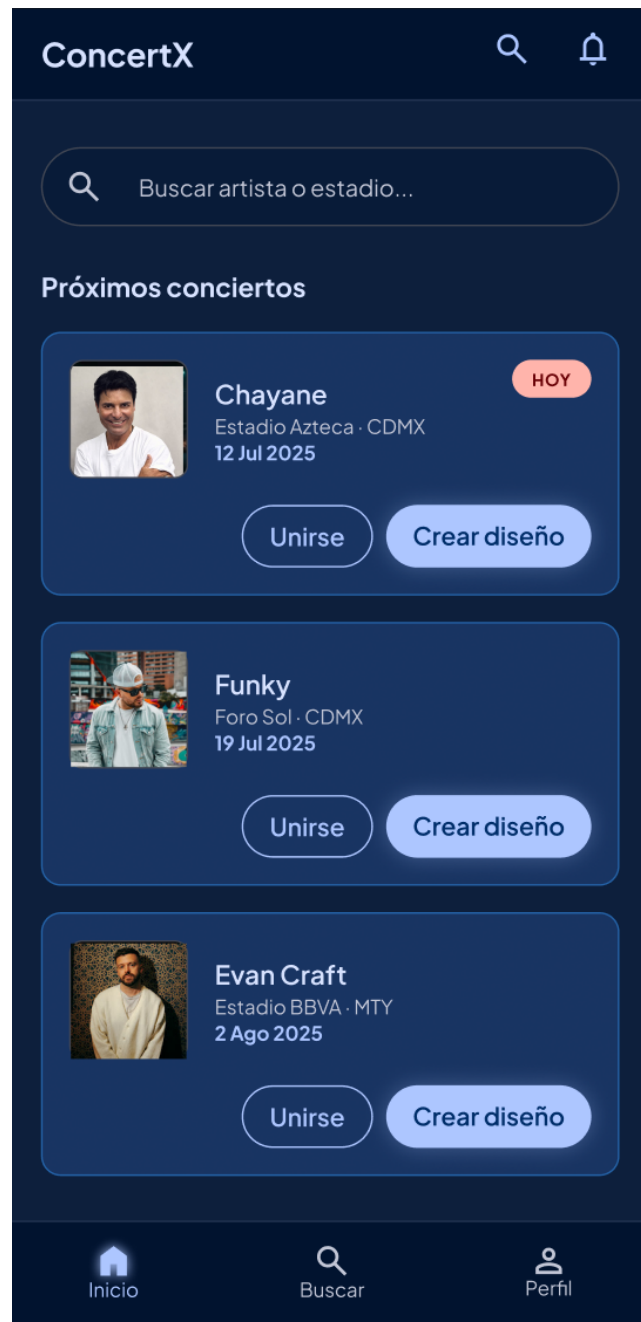


Figura 5: Vista del teléfono Android durante el concierto

- **Smartwatch:** Un único elemento en pantalla —el color o ícono del momento—. La pantalla se ve mientras el usuario mueve el brazo, por lo que menos es más.

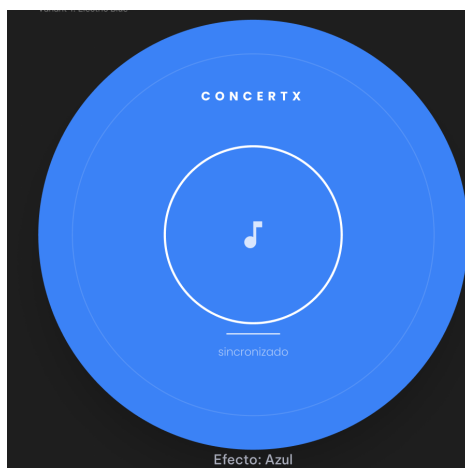


Figura 6: Vista del smartwatch con efecto activo

- **Pantalla grande:** Permite mayor densidad de información porque el usuario la observa desde lejos y no interactúa directamente con ella.



Figura 7: Vista de Smart TV con mapa de asientos y código del evento

3.6. Pruebas de Usabilidad Planeadas

Se realizarán pruebas con un mínimo de **5 participantes** antes de la entrega final:

1. **Prueba de primer uso:** El participante instala la app y llega a unirse a un evento sin recibir instrucciones previas. Se mide el tiempo y los puntos de confusión.
2. **Prueba en condiciones de oscuridad:** Se simula el ambiente de un concierto bajando el brillo al mínimo y evaluando la legibilidad de la interfaz.

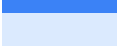
3. **Prueba con smartwatch:** Un usuario verifica si las vibraciones son perceptibles, distinguibles y no resultan molestas tras 30 minutos de uso.

Subtotal SA.2: ____ / 6 elementos

4. SA.3 — Diseño de Interfaces de Usuario

4.1. Paleta de Colores

La paleta está centrada en tonos azul marino, elegidos por su legibilidad en ambientes oscuros y su carácter profesional. Los tonos más claros aportan contraste y énfasis sin romper la coherencia visual.

Color	Nombre	Uso principal	Contraste
	Azul marino oscuro #0D1F3C	Fondos principales, pantalla del concierto	✓ AAA
	Azul marino medio #1B3A6B	Encabezados, barras de navegación	✓ AAA
	Azul marino claro #2563A8	Subtítulos, íconos activos	✓ AA
	Azul cielo #3B82F6	Botones de acción, acentos, enlaces	✓ AA
	Azul hielo #DBEAFE	Fondos de tarjetas, cajas informativas	✓ AAA

4.2. Tipografía

Fuente	Aplicación	Tamaño mínimo
Poppins Bold	Títulos de pantalla y encabezados principales	20 px
Poppins Medium	Etiquetas de botones, menús y subtítulos	14 px
Poppins Regular	Cuerpo de texto, descripciones, formularios	14 px
Source Code Pro	Códigos de evento y valores técnicos	13 px

Poppins está disponible en Google Fonts e integrada en Flutter sin dependencias adicionales, cumpliendo el mínimo de 12 px de accesibilidad en todos los casos.

4.3. Componentes Reutilizables

Se diseñan una sola vez y se usan de forma consistente en toda la aplicación. A continuación se presenta cada componente con su imagen de referencia:

1. Recuadro de código de diseño

Muestra el código alfanumérico del evento generado por el sistema. Se usa en la pantalla de código generado y en la vista de Smart TV. Tipografía monoespaciada, borde azul eléctrico, fondo oscuro para máximo contraste.



Figura 8: Componente 1 — Recuadro de código de diseño

2. Botón principal

Botón de acción primaria con fondo azul cielo (#3B82F6), texto blanco en Poppins SemiBold, altura de 52 px y esquinas redondeadas de 8 px. Se usa para las acciones más importantes de cada pantalla: Crear diseño, Unirme, Generar código, etc.

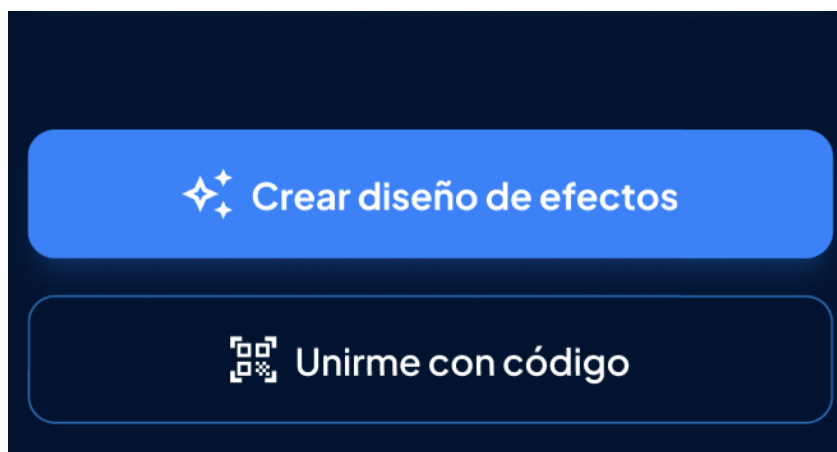


Figura 9: Componente 2 — Botón principal

3. Información del artista

Tarjeta que presenta los datos del concierto: nombre del artista, estadio, ciudad y fecha. Fondo azul marino medio (#1B3A6B), esquinas de 12 px y área de imagen de portada a la izquierda. Incluye los dos botones de acción (Crear diseño / Unirse) en la parte inferior.

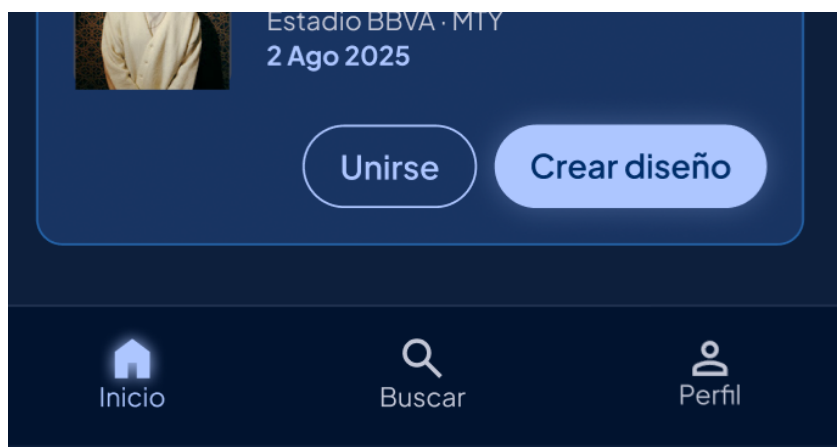


Figura 10: Componente 3 — Información del artista

4. Menú de navegación

Barra de navegación inferior con tres pestañas: Inicio, Buscar y Perfil. Fondo azul marino oscuro (#0D1F3C), borde superior de 1 px en #1B3A6B. La pestaña activa se resalta en azul cielo (#3B82F6); las inactivas en gris (#64748B).



Figura 11: Componente 4 — Menú de navegación

5. Barra de búsqueda

Campo de búsqueda full-width con ícono de lupa a la izquierda, texto placeholder en gris y fondo #1B3A6B. Se activa con borde azul al enfocar. Permite buscar conciertos por nombre de artista o estadio desde la pantalla principal.

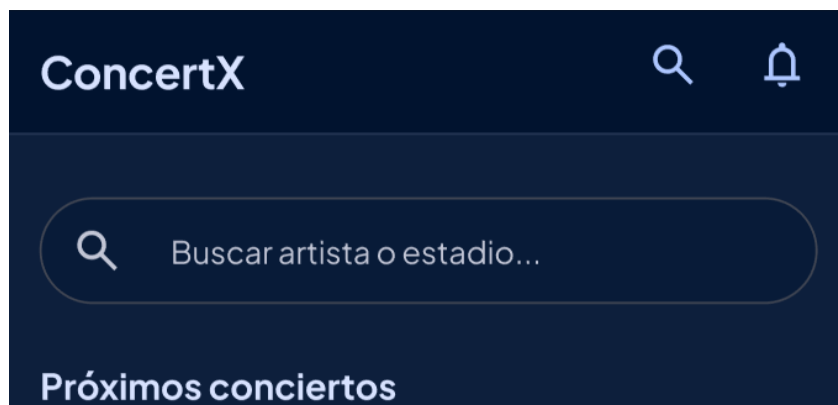


Figura 12: Componente 5 — Barra de búsqueda

Total de componentes definidos: 5

4.4. Layout por Dispositivo

Estructura vertical de tres áreas:

1. **Encabezado:** Nombre del concierto y estado de la conexión.
2. **Área central:** Efecto visual activo, ocupa el 70 % de la pantalla.
3. **Barra inferior:** Navegación entre Inicio, Buscar y Perfil.

Vista única sin menús:

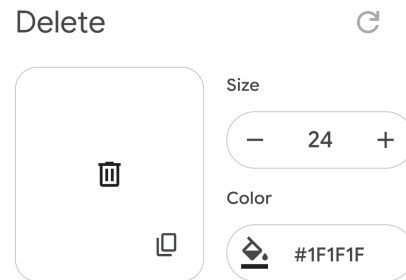
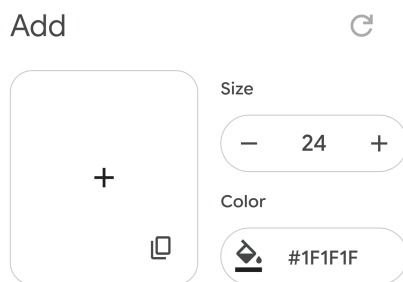
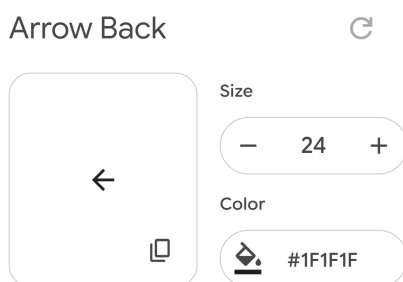
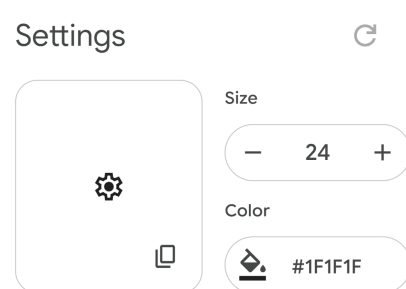
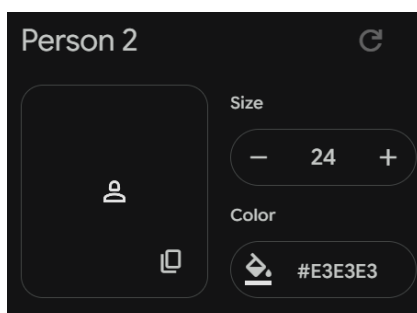
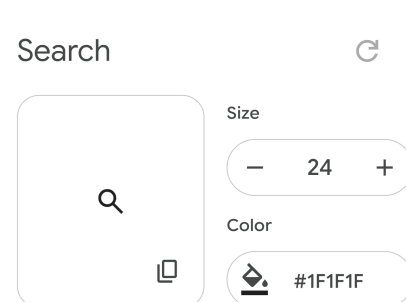
1. **Fondo completo:** Color del efecto activo cubre toda la pantalla.
2. **Centro:** Ícono o texto muy corto del momento (p. ej. *CORO*).
3. **Sin navegación:** La interacción es exclusivamente visual y háptica.

Tres columnas:

1. **Izquierda:** Contador de usuarios conectados al evento.
2. **Centro:** Mapa de asientos del estadio — azul conectado, oscuro disponible.
3. **Derecha:** Código QR y código alfanumérico del evento para unirse.

4.5. Iconografía

Se utilizará la librería **Material Symbols** de Google, disponible en Flutter mediante el paquete `material_symbols_icons`. Es gratuita, mantenida activamente por Google y licenciada bajo Apache 2.0. A continuación se muestran los íconos seleccionados para la aplicación, obtenidos directamente desde `fonts.google.com/icons`:

**refresh** — Recargar vista**delete** — Eliminar diseño**add** — Crear evento**close** — Cerrar pantalla**back** — Regresar atrás**settings** — Configuraciones**perfil** — Perfil**search** — Buscar

Librería: Material Symbols — Flutter package (`material_symbols_icons`)

4.6. Estados de Componentes

Estado	Comportamiento visual
Normal	Apariencia estándar definida en la guía de estilos
Presionado	Oscurecimiento del 10 % en el fondo, escala al 97 %
Deshabilitado	Opacidad reducida al 40 %, sin respuesta al toque del usuario
Cargando	Indicador circular sobre el componente, sin bloquear la pantalla
Error	Borde en rojo (#DC2626) y mensaje descriptivo debajo del elemento

4.7. Guía de Estilo

La guía de estilo de Concertx define las reglas visuales que deben seguirse en todas las pantallas de la aplicación. Los elementos de paleta de colores, tipografía e iconografía ya fueron documentados en las secciones anteriores. A continuación se especifican los elementos restantes:

Espaciado y Márgenes

Elemento	Valor
Margen lateral de pantalla	16 px en teléfono, 24 px en tablet
Separación entre secciones	24 px
Separación entre componentes	12 px
Padding interno de tarjetas	16 px en todos los lados
Padding interno de botones	12 px vertical · 24 px horizontal
Área táctil mínima	44 × 44 px (accesibilidad)

Bordes y Esquinas

Componente	Radio de esquina
Botones primarios y secundarios	8 px
Tarjetas de concierto	12 px
Campos de texto	6 px
Modales y diálogos	16 px
Chips / etiquetas	20 px (completamente redondeados)
Indicadores de zona en mapa	50 % (círculo completo)

Sombras y Elevación

Nivel	Aplicación
Sin sombra (nivel 0)	Fondos de pantalla, elementos planos
Sombra suave (nivel 1)	Tarjetas de concierto, campos de texto
Sombra media (nivel 2)	Botones activos, barra de navegación
Sombra alta (nivel 3)	Modales, diálogos emergentes, menús

Tono de Voz e Idioma

El texto dentro de la aplicación sigue estas reglas:

- **Lenguaje directo y breve:** Los botones usan verbos en infinitivo: *Crear, Unirse, Buscar, Guardar, Salir.*
- **Segunda persona informal:** Se habla al usuario de «tú»: *«Ingresa el código del evento»,* nunca *«El usuario debe ingresar...»*
- **Mensajes de error descriptivos:** Indican exactamente qué pasó y cómo resolverlo: *«El código no existe. Verifica que lo hayas escrito correctamente.»*
- **Sin tecnicismos en la UI:** Términos como «WebSocket» o «JWT» no aparecen en pantallas visibles para el usuario final.

Comportamiento de Animaciones

Tipo de animación	Duración y curva
Transición entre pantallas	250 ms · curva easeInOut
Aparición de modales	200 ms · curva easeOut
Cambio de color de efecto	150 ms · curva linear (concierto)
Vibración del smartwatch	Instantánea al recibir señal
Indicador de carga	Rotación continua a 1 vuelta/s

Subtotal SA.3: ___ / 7 elementos

5. SA.4 — Seguridad Transversal

5.1. Leyes Aplicables

- **LFPDPPP** — Ley Federal de Protección de Datos Personales en Posesión de Particulares (México). Aplica directamente porque la aplicación recopila nombre, correo, número de teléfono y contraseña de los usuarios.
- **Código de Comercio** — Relevante si en versiones futuras se incorporan pagos o transacciones electrónicas dentro de la plataforma.

5.2. Datos Personales Identificados

La aplicación recopila únicamente los datos necesarios para el funcionamiento del sistema:

- **Nombre completo** — Para identificar al usuario en el registro y en el perfil del evento.
- **Correo electrónico** — Para autenticación, recuperación de contraseña y comunicaciones del sistema.
- **Número de teléfono** — Dato de contacto adicional, solicitado en el registro.
- **Contraseña** — Almacenada con hash seguro (bcrypt), nunca en texto plano.

5.3. Justificación de Seguridad

Estos datos exigen protección por razones concretas. El correo y el número de teléfono son datos personales identificables que, de quedar expuestos, podrían usarse para contactar o localizar a una persona sin su consentimiento. La contraseña, si no se cifra correctamente, permite el acceso no autorizado a la cuenta y, en consecuencia, a la información del usuario dentro del sistema.

5.4. Plan de Almacenamiento

El sistema ofrece dos opciones de infraestructura equivalentes en seguridad:

Dato	Forma de almacenamiento	Opción A	Opción B
Credenciales (email, hash pass, teléfono)	Tabla de usuarios en PostgreSQL con cifrado bcrypt	Supabase	Azure PostgreSQL
Perfil y datos del evento	Tablas relacionales: usuarios, eventos, diseños, asientos	Supabase	Azure PostgreSQL
Autenticación y tokens JWT	Generados por Express.js, firmados con clave secreta de entorno	Supabase Auth	Azure AD / JWT propio
Efectos en tiempo real	En memoria del servidor vía WebSockets, sin persistencia	Supabase Realtime	Azure SignalR
Sesión activa en dispositivo	Token JWT en almacenamiento seguro local (Flutter Secure Storage)	Ambas opciones	

Ambas opciones proveen conexiones cifradas SSL/TLS a PostgreSQL, respaldos automáticos y paneles de monitoreo sin configuración adicional.

5.5. Comunicaciones Seguras

Toda comunicación entre la app, el smartwatch y el backend Express.js se realiza sobre **HTTPS con TLS 1.2 o superior**. Las conexiones WebSocket operan sobre el protocolo seguro **WSS** (WebSocket Secure). Ningún dato viaja en texto plano entre cliente y servidor.

✓ **Confirmado**

5.6. Gestión de Credenciales y API Keys

Las claves de acceso a bases de datos y servicios externos **no se escriben en el código fuente bajo ninguna circunstancia**:

- **Backend Express.js:** Variables de entorno en archivo `.env` excluido del repositorio mediante `.gitignore`.
- **Flutter (Android):** Claves pasadas con `-dart-define` en tiempo de compilación o mediante el paquete `flutter_dotenv`, igualmente excluido del control de versiones.
- **Repositorio Git:** Los archivos `.env` y cualquier archivo de configuración con credenciales están listados en `.gitignore` desde el inicio del proyecto.

5.7. Validación de Entrada

Antes de procesar cualquier dato ingresado por el usuario, el sistema verifica:

- **Correo electrónico:** Formato válido con expresión regular.
- **Contraseña:** Mínimo 8 caracteres, al menos una letra mayúscula y un número.
- **Número de teléfono:** Solo dígitos, longitud entre 10 y 13 caracteres.
- **Código de evento:** Solo caracteres alfanuméricos, sin espacios ni símbolos.
- **Selección de asiento:** Valores dentro del rango configurado para ese evento.

Esta validación ocurre en **dos capas:** primero en el cliente Flutter antes de enviar la solicitud, y luego en el backend Express.js antes de interactuar con la base de datos.

5.8. Aviso de Privacidad

El siguiente texto corresponde al aviso de privacidad que el usuario verá dentro de la aplicación antes de completar su registro. Debe aceptarlo de forma explícita para continuar.

Aviso de Privacidad — Concertx

¿Quién es responsable de tus datos?

Concertx es la plataforma responsable de la información que compartes al registrarte. Si tienes alguna duda o solicitud relacionada con tus datos, puedes escribirnos a privacidad@concertx.app.

¿Qué información recopilamos?

Al crear tu cuenta solicitamos tu nombre completo, correo electrónico, número de teléfono y una contraseña. Durante un evento también registramos tu zona, fila y número de asiento dentro del recinto.

¿Para qué usamos tu información?

Tus datos se utilizan exclusivamente para: crear y gestionar tu cuenta, permitirte unirse a eventos o crear diseños de efectos, y sincronizar tu dispositivo con los demás asistentes en tiempo real.

¿Compartimos tu información?

No compartimos tus datos con terceros con fines comerciales o publicitarios. Únicamente se transmiten a nuestra infraestructura de nube (Supabase o Microsoft Azure) como parte técnica del servicio, quienes cuentan con sus propias certificaciones de seguridad.

¿Cómo protegemos tus datos?

Tu contraseña se almacena cifrada y nunca se guarda en texto legible. Toda la comunicación entre la app y nuestros servidores viaja cifrada. Nadie en el equipo de Concertx tiene acceso a tu contraseña.

¿Cuánto tiempo conservamos tu información?

Tus datos permanecen activos mientras uses la plataforma. Si no registras actividad durante 12 meses, tu cuenta y la información asociada serán eliminadas. Puedes solicitar la eliminación de tu cuenta en cualquier momento desde la app o escribiéndonos.

Tus derechos

Puedes acceder, corregir o solicitar la eliminación de tus datos en cualquier momento. Solo escríbenos a privacidad@concertx.app y respondemos en un máximo de 20 días hábiles.

Acepto el Aviso de Privacidad

No acepto

5.9. Certificación de Acceso para Pruebas

Antes de cada prueba, se le explica al participante de forma breve lo siguiente:

1. **Qué se va a probar:** Qué es Concertx y qué va a hacer durante la sesión, que dura máximo 30 minutos.
2. **Qué se observa:** Cómo usa la app y qué le resulta confuso.
3. **Es voluntario:** Puede detener la prueba cuando quiera.
4. **Qué pasa después:** La cuenta de prueba se elimina al terminar.

Se requiere la participación de al menos **5 personas** para validar la usabilidad de la aplicación.